

图像理论基础 实验指导手册



1.图像理论基础	3
1.1 实验任务	3
1.2 实验背景	3
1.3 实验介绍	3
1.3.1 图像的基础理论介绍	3
1.3.2 OpenCV 介绍	3
1.4 实验流程图	3
1.5 实验原理	4
1.5.1 关键代码	5
1.5.2 代码详解	7
1.6 实验步骤	8
1.6.1 源码位置	8
1.6.2 运行步骤	8

1. 图像理论基础

1.1 实验任务

- 掌握 OpenCV 实现对图的读取、显示和保存方法。
- 掌握 OpenCV 实现对视频的读取、显示方法。
- 掌握 OpenCV 对各种视频流设备的调用。
- 掌握 OpenCV 对图像常用属性信息的获取。

1.2 实验背景

本实验是 OpenCV 计算机视觉入门的基础实操实验，聚焦图像与视频流的最基础、最核心操作，是后续 AI 图像视觉开发、目标检测、图像处理等进阶内容的前置必备技能，主要面向计算机视觉、嵌入式视觉开发的初学者。

1.3 实验介绍

1.3.1 图像的基础理论介绍

假设当前有一张图片，图片里面有一朵花，人类的视觉在很多不同的环境下都能很轻易地从这幅图像中识别花朵，这得益于我们眼睛的强大功能。但是，计算机视觉并不会像人类视觉那样能够对图像进行感知和识别，更不会自动控制焦距和光圈，而是把图像解析为按照栅格状排列的数字。这种数字又被称为像素点，故在计算机中图像是由像素点构成的，像素点的数值范围为 $0 \sim 255$ ，表示该像素点的亮度，0 表示黑，255 表示白。

图像分为单通道图（灰度图）和多通道图（彩色图），彩色图像类型为 RGB，它是由红（R）、绿（G）、蓝（B）三种基本颜色通道组合成不同的颜色。假设彩色图像的长和高是 500×500 ，则这个彩色图像的最终维度是 $500 \times 500 \times 3$ ，而灰度图只由一个通道表示亮度即可，故长和高是 500×500 的灰度图像，它的最终维度是 500×500 。

1.3.2 OpenCV 介绍

OpenCV 是一个开源发行的跨平台计算机视觉和机器学习软件库，可以运行在 linux、Windows、Android 和 MAC OS 操作系统上。它轻量且高效——由一系列 C 函数和少量 C++ 类构成，同时提供了 Python、Ruby、MATLAB 等语言的接口，实现了图像处理和计算机视觉方面的很多通用算法。

OpenCV 用 C++ 语言编写，它具有 C++，Python，java 和 MATLAB 等接口。OpenCV 自发布起便得到广泛应用，其中包括在安保以及工业检测系统，网络产品以及科研工作，医学、卫星和网络地图（例如，医学图像的降噪，街景图像或者航空图像的拼接及其扫描校准等），汽车自动驾驶，相机校正等。此外，OpenCV 还被应用到处理声音的频谱图像上，进而实现对声音的识别。

1.4 实验流程图



1.5 实验原理

1.5.1 关键代码

1.5.1.1 读取、显示和保存图像

OpenCV 通过使用 `imread()` 方法进行读取图像。

```
image = cv2.imread(filename, [flags])
```

```
...
```

image: 返回的是读取到的图像。注意：OpenCV 读取的图像是 BGR 的方式，即 **image** 的三通道顺序为 BGR

filename: 要读取的图像的完整文件名。

flags: 读取图片的方式，可选项。

`cv2.IMREAD_COLOR` 或 **1**: 始终将图像转换为 3 通道 BGR 彩色图像，默认方式

`cv2.IMREAD_GRAYSCALE` 或 **0**: 始终将图像转换为单通道灰度图像

`cv2.IMREAD_UNCHANGED` 或 **-1**: 按原样返回加载的图像（使用 Alpha 通道）

`cv2.IMREAD_ANYDEPTH` 或 **2**: 在输入具有相应深度时返回 16 位/ 32 位图像，否则将其转换为 8 位

`cv2.IMREAD_ANYCOLOR` 或 **4**: 以任何可能的颜色格式读取图像

```
...
```

OpenCV 提供了 `imshow()` 方法进行图像的显示，但是在使用的过程中往往搭配用于等待用户按下键盘上按键的时间的 `waitKey()` 方法和用于销毁所有正在显示图像的窗口的 `destroyAllWindows()` 方法一起使用。

```
cv2.imshow(name, image)
```

```
...
```

`imshow()` 方法无返回值

name: 显示图像的窗口名称。

image: 要显示的图像。

```
...
```

```
ret = cv2.waitKey(value)
```

```
...
```

ret: 与被按下的按键对应的 ASCII 码。例如，Esc 键的 ASCII 码是 27，当用户按 Esc 键时，`waitKey()` 方法的返回值是 27。如果没有按键被按下，`waitKey()` 方法的返回值是 -1。设置按下 Esc 键停止的方法为：`if cv2.waitKey(1) & 0xFF == 27: break` 结合视频流读取的时候当按下 Esc 即结束。注意：在后面的 OpenCV 实验中均是以 Esc 键停止 OpenCV 程序。

value: 等待用户按下键盘上按键的时间，单位为毫秒 (ms)。当 **value** 的值为负数、0 或者空时，表示无限等待用户按下键盘上按键的时间。

```
...
```

```
cv2.destroyAllWindows()
```

```
...
```

`destroyAllWindows()` 方法无任何参数和返回值

```
...
```

OpenCV 使用 `imwrite()` 方法用于按照指定路径保存图像

```
cv2.imwrite(filename, image)
```

```
...
```

filename: 保存图像时所用的完整路径。

image: 要保存的图像。

```
...
```

1.5.1.2 读取、显示视频流

视频是由很多帧静止的图像组成，随着时间轴图像连续切换。帧数慢，图像不连贯，肉眼可见的图像变换，就会觉得卡。帧数高，图像连贯，肉眼看不出图像的切换。

视频流资源的读取是一个连贯的过程，OpenCV 提供了 VideoCapture 类的相关方法来读取视频流资源。VideoCapture 类提供了构造方法 VideoCapture()，用于完成视频资源的初始化工作。

```
capture = cv2.VideoCapture(value)
```

```
...
```

capture: 要打开的摄像头。

value: 摄像头的设备索引或打开视频的文件名（.avi 文件）。

由于摄像头的数量及其设备索引的先后顺序由操作系统决定。以 USB 摄像头为例，当插入到 win10 电脑上，可以通过设备索引为 0 来进行调用，但是当插入到 LPPE002 实验箱上时，则要指定设备索引为 10 来进行调用。

```
...
```

为了检验摄像头或视频资源初始化是否成功，使用 VideoCapture 类提供了 isOpened() 方法进行检验。

```
ret = capture.isOpened()
```

```
...
```

ret: isOpened() 方法的返回值。如果摄像头初始化成功，ret 的值为 True；否则，ret 的值为 False。

```
...
```

摄像头初始化后，可以使用 VideoCapture 类提供了 read() 方法从视频流中读取帧。

```
ret, image = capture.read()
```

```
...
```

ret: 是否读取到帧。如果读取到帧，ret 的值为 True；否则，ret 的值为 False。

image: 读取到的帧。因为帧指的是构成视频的图像，所以可以把“读取到的帧”理解为“读取到的图像”。

```
...
```

由于视频是由很多帧静止的图像组成，故视频流的显示即是每一帧的图像显示出来，即将每一帧的 image 都利用 imshow() 显示出来。

在不需要视频流时，使用 VideoCapture 类提供的 release() 方法关闭视频流。

```
capture.release()
```

在图像视觉开发的过程中往往还会对 VideoCapture 返回的 capture 进行其他 set 操作，但是这里仅介绍基础且常用的操作故需要读者自行进行积累。

在读取、显示视频流的时候，常常会使用 cap.set 进行设置视频捕获对象的属性。并且在 OpenCV 中，属性由整数常量表示，这些常量通常在 cv2 模块中以整数的形式定义。故设置视频帧的宽度和高度属性的代码为：

```
cap.set(3,num1) 设置视频帧的宽度为 num1 像素。
```

```
cap.set(4,num2) 设置视频帧的高度为 num2 像素。
```

1.5.1.3 LPPE002 实验箱调用各种视频流设备

V4L2 (Video for Linux two) 为 linux 下视频设备程序提供了一套接口规范。包括一套数据结构和底层 V4L2 驱动接口。只能在 linux 下使用。它使程序有发现设备和操作设备的能力。它主要是用一系列的回调函数来实现这些功能。例如设置摄像头的频率、帧频、视频压缩格式和图像参数等等。当然也可以用于其他多媒体的开发，如音频等。

在 Linux 下，所有外设都被看成一种特殊的文件，称为“设备文件”，可以像访问普通文件一样对其进行读写。一般来说，采用 V4L2 驱动的摄像头设备文件是 /dev/video*。V4L2 支持两种方式来采集图像：内存映射方式(mmap)和直接读取方式(read)。

V4L2 规范中不仅定义了通用 API 元素(Common API Elements)，图像的格式(Image Formats)，输入/输出方法(Input/Output)，还定义了 Linux 内核驱动处理视频信息的一系列接口(Interfaces)，这些接口主要有：

1. 视频采集接口——Video Capture Interface;
2. 视频输出接口—— Video Output Interface;
3. 视频覆盖/预览接口——Video Overlay Interface;
4. 视频输出覆盖接口——Video Output Overlay Interface;
5. 编解码接口——Codec Interface。

调用 RTSP 摄像头，只需要正确的输入用户名和密码以及 rtsp 的地址，若读者上手没有 rtsp 设备，可以在手机上安装 IP 摄像头进行尝试。

```
cap = cv2.VideoCapture("rtsp://admin:admin@192.168.1.89/av_stream")
```

1.5.1.4 常用图像操作

在 AI 图像视觉开发的过程中，经常需要获取图像的大小、维度等图像属性。

```
image.shape
```

```
'''
```

shape: 如果是彩色图像，那么获取的是一个包含图像的水平像素、垂直像素和通道数的数组，即（垂直像素，水平像素，通道数）；如果是灰度图像，那么获取的是一个包含图像的水平像素和垂直像素的数组，即（垂直像素，水平像素）。

```
'''
```

```
image.size
```

```
'''
```

size: 获取的是图像包含的像素个数，其值为“水平像素×垂直像素×通道数”。灰度图像的通道数为 1。

```
'''
```

1.5.2 代码详解

● 图像读取、显示和保存代码

```
import cv2 # 导入 OpenCV 库
```

```
image = cv2.imread("./img/cat.jpg") # 读取图像
```

```
cv2.imshow("res", image) # 显示图像
```

```
cv2.waitKey(0) # 按任意键结束
```

```
cv2.destroyAllWindows() # 销毁显示窗口
```

```
cv2.imwrite("./cat_save.jpg", image) # 将图像保存在当前目录下，并命名为 cat_save.jpg
```

● 视频读取、显示代码

```
import cv2 # 导入 OpenCV 库
```

```
capture = cv2.VideoCapture(0) # 打开摄像头，在实验箱上索引 0 为实验箱的 mipi 摄像头索引号
```

```
while (capture.isOpened()): # 摄像头被打开后
```

```
    ret, image = capture.read() # 从摄像头中实时读取视频
```

```
    cv2.imshow("Video", image) # 在窗口中显示读取到的视频
```

```
    if cv2.waitKey(1) & 0xFF == ord('q'): break # 按下 q 键停止
```

```
capture.release() # 关闭实验箱内置摄像头
```

```
cv2.destroyAllWindows() # 销毁显示摄像头视频的窗口
```

● 图像属性代码

```
import cv2
```

```
image = cv2.imread("./img/cat.jpg") # 导入图像
```

```
print(image.shape) # 获取图像维度
```

```
print(image.size) # 获取图像包含的像素个数
```

注意：程序中如果出现 `cv2.waitKey(0)` 则可以按 LPPE002 实验箱键盘上任意键结束程序，若出现 `if`

cv2.waitKey(1) & 0xFF == ord('q') 则需要按键盘上的 Q 键结束程序。

1.6 实验步骤

1.6.1 源码位置

实验源码：源码/ch01_opencv_base/section02_OpenCV_img_base

目录结构：

```
├── 01_image_base.py
├── 02_cam_base.py
├── 03_cam_device.py
├── 04_image_attribute.py
├── img
│   └── cat.jpg
```

1.6.2 运行步骤

Step1:实验箱准备，具体方法参考操作指南的“实验手册\操作指南\01-实验前准备和结束收纳”

Step2:使用 SSH 远程登录到实验平台,具体方法参考操作指南的“实验手册\操作指南\03-远程登录到实验平台”

Step3:使用 FTP 将源码上传到实验平台,具体方法参考操作指南的“实验手册\操作指南\04-上传下载文件到实验平台”

Step4:输入以下命令启动 conda 虚拟实验环境:

```
conda activate py37
```

Step5:输入以下命令运行代码

```
cd ch01_opencv_base/section02_OpenCV_img_base
```

```
Python 01_image_base.py
```

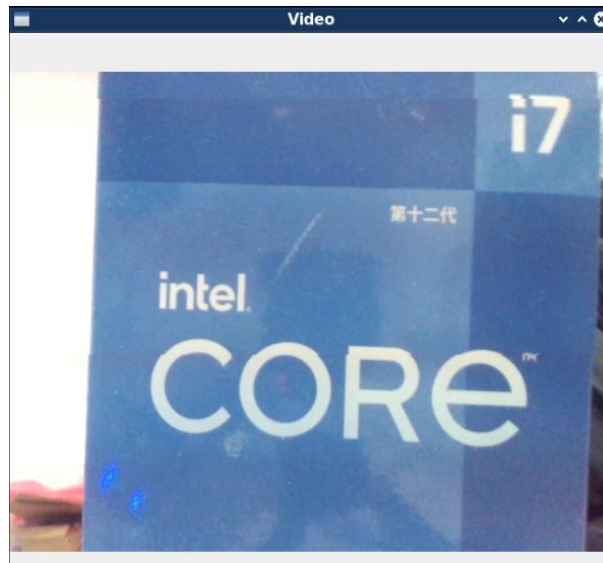
实验结果如下：



```
Python 02_cam_device.py
```

注意：需要接入摄像头，后面的实验中要注意是否需要接入摄像头，并且代码中的相机设备号为：
"/dev/video10"，每个机器的情况是不一样的，最好需要自己使用命令：**ls /dev/video***来查看；

实验结果如下：



Python 03_image_attribute.py

实验结果如下：

```
(360, 480, 3)
518400
```